

Checklist:

Team Topologies – Evolve Team Structures

1. Map the Current Team System

- ☐ Identify each team's primary topology: stream-aligned, platform, enabling, or complicated-subsystem.
- ☐ Map the key team-to-team interactions.
- ☐ Label each interaction mode: collaboration, X-as-a-Service, or facilitating.
- ☐ Check whether each interaction mode matches the work being done right now.
- ☐ Identify unclear ownership, blurred boundaries, or hidden dependencies.

2. Decide How Much Collaboration Is Needed

- ☐ Confirm that every ongoing collaboration has a clear purpose.
- ☐ Classify the purpose of collaboration as one of:
 - ☐ Valuable discovery
 - ☐ Valuable capability building
 - ☐ Valuable deficiency-fixing
- ☐ Remove or reduce collaboration that is not clearly valuable.
- ☐ Timebox collaboration where possible.
- ☐ Define exit criteria for moving from collaboration to a clearer service boundary.
- ☐ Watch for collaboration becoming a permanent substitute for better APIs, platforms, or ownership.

3. Move Toward X-as-a-Service When Appropriate

- ☐ Identify services, platforms, or capabilities that other teams should consume with low effort.
- ☐ Define clear APIs, interfaces, documentation, and support expectations.

- ☐ Reduce the cognitive load required for consuming teams.
- ☐ Ensure the providing team owns the reliability and usability of the service.
- ☐ Move from close collaboration to limited collaboration once discovery is complete.
- ☐ Move to X-as-a-Service when the boundary is stable and predictable delivery matters most.

4. Use Collaboration for Learning and Adoption

- ☐ Use temporary collaboration when one team has valuable practices another team needs.
- ☐ Pair teams to accelerate adoption of new tools, testing practices, automation, or platform capabilities.
- ☐ Use collaboration to expose hidden constraints, such as hardware limits, testing gaps, or operational complexity.
- ☐ Reassess after the learning period to decide whether to:
 - ☐ Continue collaboration
 - ☐ Create a platform/service model
 - ☐ Reorganize ownership
 - ☐ Split or merge teams

5. Watch for Triggers That Team Topologies Need to Evolve

Software has grown too large for one team

- ☐ The product or system has grown beyond what one team can understand well.
- ☐ Other teams wait on one team for changes.
- ☐ Specific changes are repeatedly assigned to the same people.
- ☐ Team members complain about missing or poor documentation.
- ☐ Bottlenecks are forming around “who knows what.”

Delivery cadence is slowing

- ☐ Releases feel slower than they used to.
- ☐ Velocity or throughput is trending downward compared with the previous year.
- ☐ Teams complain that the delivery process has become more complex.
- ☐ Work in progress is increasing.
- ☐ Changes are waiting for another team’s action.
- ☐ New dependencies, handoffs, or silos are reducing autonomy.

Many business services rely on many lower-level services

- ☐ Stream-aligned teams lack end-to-end visibility.
- ☐ Integration across subsystems is slowing flow.
- ☐ Reusing shared services or subsystems is becoming harder.
- ☐ Teams cannot easily diagnose where problems occur.
- ☐ Lower-level systems lack consistent telemetry, logging, or correlation IDs.

6. Platformize Where It Improves Flow

- ☐ Identify lower-level services or APIs that many teams depend on.
- ☐ Decide whether these should become a platform capability.
- ☐ Provide a consistent developer experience for consuming teams.
- ☐ Add documentation, self-service access, health checks, test harnesses, and service-level expectations.
- ☐ Create useful telemetry: logs, metrics, dashboards, correlation IDs, and tracing.
- ☐ Consider an “outer platform” or wrapper when lower-level services are inconsistent or fragmented.
- ☐ Make the platform easier to consume than building custom one-off solutions.

7. Build Organizational Sensing

- ☐ Treat teams and team interactions as signals about organizational health.
- ☐ Regularly ask:
 - ☐ Are users behaving differently than we expected?
 - ☐ Do we need to change team interaction modes?
 - ☐ Should we still build this in-house, or use an external provider?
 - ☐ Is collaboration between teams still effective?
 - ☐ Should a team move toward X-as-a-Service?
 - ☐ Is work flowing smoothly through each team?
 - ☐ Does the platform provide what consuming teams actually need?
 - ☐ Are promises between teams still realistic and achievable?
- ☐ Use operational data, customer signals, team feedback, and delivery metrics together.
- ☐ Look for weak signals before they become major structural problems.

8. Treat Operations as Feedback to Development

- ☐ Ensure operational experience feeds directly into development decisions.
- ☐ Avoid separating “new work” teams from “maintenance” or BAU teams in a way that breaks learning.

- ☐ Give teams responsibility for both new services and existing systems where practical.
- ☐ Use production telemetry to improve design, reliability, usability, and security.
- ☐ Staff operations or support work with experienced people who can recognize and explain system issues.
- ☐ Make operational pain visible to product and engineering teams.

9. Review and Evolve Regularly

- ☐ Review team boundaries every few months, not every day or week.
- ☐ Check whether current topology still fits product, technology, and market conditions.
- ☐ Expect different parts of the organization to require different interaction modes simultaneously.
- ☐ Allow some teams to be in discovery while others consume services predictably.
- ☐ Make interaction modes explicit so people understand why teams work together differently.
- ☐ Adjust structures, APIs, platforms, and responsibilities as the organization learns.

10. Final Health Check

- ☐ Teams have clear purposes.
- ☐ Team interactions are explicit.
- ☐ Collaboration is intentional, valuable, and limited where possible.
- ☐ Platforms reduce cognitive load for stream-aligned teams.
- ☐ Operations provide useful sensory input to development.
- ☐ Delivery flow is measured and improving.
- ☐ The organization can sense, adapt, and reshape itself as conditions change.